Accessible First

Technical Foundations (Phase 0 and Phase 1)

Language

Primary language:

English

Everything must be written in English:

- code
- comments
- documentation
- commit messages
- examples
- error messages

Localization should support multiple languages, but English is the source language.

Core Technologies

TypeScript

Primary language:

TypeScript

Reasons:

- excellent tooling;
- strong typing;
- maintainability;
- self-documenting code;
- huge ecosystem;
- no runtime cost.

TypeScript should be used everywhere.

Native HTML First

Prefer native HTML whenever possible.

Examples:

button instead of div

fieldset instead of custom groups

details where appropriate

dialog when suitable

No ARIA is better than bad ARIA.

CSS

Plain CSS.

Prefer:

CSS variables

Flexbox

Grid

Logical properties

Avoid:

CSS frameworks

Heavy preprocessors

Unnecessary abstractions

JavaScript Philosophy

Minimal JavaScript.

Progressive enhancement.

Performance first.

Accessibility first.

Framework Independence

The Core must not depend on React.

Everything should be built around independent logic.

Framework adapters may be added later.

React

Vue

Svelte

Web Components

Solid

Future frameworks

Architecture

packages/

apps/

docs/

examples/

playground/

tests/

Packages

accessible-first-core

Pure logic.

No UI.

No framework dependencies.

---

accessible-first-components

UI components.

---

accessible-first-hooks

Reusable behavior.

---

accessible-first-utils

Utilities.

---

accessible-first-icons

Icons.

---

accessible-first-themes

Themes.

---

accessible-first-locale

Localization.

---

accessible-first-testing

Accessibility testing helpers.

Development Principles

Accessibility First

Accessibility is not optional.

Native First

Use browser capabilities whenever possible.

Progressive Enhancement

Everything should work without JavaScript whenever possible.

Performance First

Avoid unnecessary abstractions.

Avoid heavy dependencies.

Stable API

Breaking changes should be rare.

Simplicity First

Good solutions should be easier than bad solutions.

Learn Before Build

Understand the problem before creating abstractions.

Need Driven Development

Components are created because they are needed in real projects.

Never because they look useful.

Initial Repository Structure

accessible-first/

docs/

examples/

playground/

packages/

accessible-first-core/

accessible-first-components/

accessible-first-hooks/

accessible-first-utils/

tests/

.github/

Coding Standards

Language:

English

Naming:

camelCase

PascalCase

kebab-case

Documentation:

Markdown

Code style:

ESLint

Formatting:

Prettier

Testing:

Vitest

Accessibility tests:

axe-core

E2E:

Initial Goal

Build a foundation that is:

simple,

stable,

accessible,

performant,

maintainable,

and independent from any specific framework.

Long-term vision matters more than short-term speed.